

Android 列表实现及第三方库推荐

Android 列表实现（Java+XML）：下拉刷新+上拉加载 & 第三方库

Android 列表核心需求是「基础展示」+「下拉刷新」+「上拉加载更多」，以下用 **Java 代码**（原生无依赖）详细讲解核心实现，再推荐常用第三方库，逻辑极简、直接可用。

一、原生实现（Java+XML，无第三方依赖）

基于 AndroidX 自带的 `RecyclerView`（列表核心）+ `SwipeRefreshLayout`（下拉刷新），上拉加载通过监听滑动事件实现，无需额外导入库。

1. 确认依赖（默认已包含）

新建 Android 项目（AndroidX 架构）后，`build.gradle` 已自带以下依赖（无需手动加）：

Code block

```
1 dependencies {
2     // RecyclerView: 列表核心控件
3     implementation 'androidx.recyclerview:recyclerview:1.3.2'
4     // SwipeRefreshLayout: 下拉刷新容器
5     implementation 'androidx.swiperefreshlayout:swiperefreshlayout:1.1.0'
6 }
```

2. 布局文件（XML）

（1）主布局：SwipeRefreshLayout 包裹 RecyclerView

`activity_main.xml`：用下拉刷新容器包裹列表，实现下拉功能：

Code block

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.swiperefreshlayout.widget.SwipeRefreshLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     android:id="@+id/swipe_refresh"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent">
7
8     <!-- 列表控件: RecyclerView -->
```

```

9      <androidx.recyclerview.widget.RecyclerView
10          android:id="@+id/recycler_view"
11          android:layout_width="match_parent"
12          android:layout_height="match_parent"/>
13
14  </androidx.swiperefreshlayout.widget.SwipeRefreshLayout>

```

(2) 列表项布局：单个条目样式

`item_list.xml`：列表中每个条目的布局（仅一个文本，简单明了）：

Code block

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <TextView
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="60dp"
6      android:gravity="center"
7      android:textSize="18sp"
8      android:background="#f5f5f5"
9      android:layout_margin="2dp"/>

```

3. Java 核心逻辑（MainActivity+Adapter）

(1) 列表适配器（MyAdapter）：渲染列表数据

RecyclerView 必须配合 Adapter 才能展示数据，封装重复的视图绑定逻辑：

Code block

```

1  import android.view.LayoutInflater;
2  import android.view.View;
3  import android.view.ViewGroup;
4  import android.widget.TextView;
5  import androidx.annotation.NonNull;
6  import androidx.recyclerview.widget.RecyclerView;
7  import java.util.List;
8
9  // 适配器：泛型参数（ViewHolder类型，数据类型）
10 public class MyAdapter extends RecyclerView.Adapter<MyAdapter.MyViewHolder> {
11
12     // 列表数据（外部传入）
13     private List<String> mData;
14
15     // 构造方法：接收数据

```

```

16     public MyAdapter(List<String> data) {
17         this.mData = data;
18     }
19
20     // 1. 创建ViewHolder (加载条目布局)
21     @NonNull
22     @Override
23     public MyViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int
viewType) {
24         View itemView = LayoutInflater.from(parent.getContext())
25             .inflate(R.layout.item_list, parent, false);
26         return new MyViewHolder(itemView);
27     }
28
29     // 2. 绑定数据到ViewHolder (每个条目显示对应数据)
30     @Override
31     public void onBindViewHolder(@NonNull MyViewHolder holder, int position) {
32         String item = mData.get(position);
33         holder.tvItem.setText(item); // 给TextView设置文本
34     }
35
36     // 3. 获取列表条目总数
37     @Override
38     public int getItemCount() {
39         return mData == null ? 0 : mData.size();
40     }
41
42     // ViewHolder: 持有条目布局中的控件 (这里是TextView)
43     static class MyViewHolder extends RecyclerView.ViewHolder {
44         TextView tvItem;
45
46         public MyViewHolder(@NonNull View itemView) {
47             super(itemView);
48             tvItem = itemView.findViewById(R.id.tv_item); // 绑定条目布局中的
TextView
49         }
50     }
51
52     // 辅助方法: 更新列表数据 (下拉刷新/上拉加载后调用)
53     public void updateData(List<String> newData) {
54         mData.clear();
55         mData.addAll(newData);
56         notifyDataSetChanged(); // 通知列表刷新
57     }
58
59     // 辅助方法: 添加更多数据 (上拉加载后调用)
60     public void addData(List<String> moreData) {

```

```
61         mData.addAll(moreData);
62         notifyDataSetChanged();
63     }
64 }
```

(2) MainActivity：初始化+下拉刷新+上拉加载

核心逻辑：初始化控件、模拟数据加载、监听下拉/上拉事件：

Code block

```
1  import android.os.Handler;
2  import android.os.Looper;
3  import android.view.View;
4  import androidx.appcompat.app.AppCompatActivity;
5  import androidx.recyclerview.widget.LinearLayoutManager;
6  import androidx.recyclerview.widget.RecyclerView;
7  import androidx.swiperefreshlayout.widget.SwipeRefreshLayout;
8  import android.os.Bundle;
9  import java.util.ArrayList;
10 import java.util.List;
11
12 public class MainActivity extends AppCompatActivity {
13
14     private SwipeRefreshLayout swipeRefresh; // 下拉刷新容器
15     private RecyclerView recyclerView;      // 列表控件
16     private MyAdapter myAdapter;           // 列表适配器
17     private List<String> dataList;          // 列表数据
18     private int page = 1;                  // 分页标识 (1=第一页)
19     private boolean isLoading = false;     // 防止重复加载
20     private boolean isLastPage = false;    // 是否是最后一页
21
22     @Override
23     protected void onCreate(Bundle savedInstanceState) {
24         super.onCreate(savedInstanceState);
25         setContentView(R.layout.activity_main);
26
27         // 1. 初始化控件
28         initView();
29         // 2. 初始化数据 (加载第一页)
30         initData();
31         // 3. 设置下拉刷新监听
32         setRefreshListener();
33         // 4. 设置上拉加载监听
34         setLoadMoreListener();
35     }
36 }
```

```
37 // 初始化控件
38 private void initView() {
39     swipeRefresh = findViewById(R.id.swipe_refresh);
40     recyclerView = findViewById(R.id.recycler_view);
41
42     // 设置列表布局：线性布局（垂直方向）
43     recyclerView.setLayoutManager(new LinearLayoutManager(this));
44     // 初始化数据集合和适配器
45     dataList = new ArrayList<>();
46     myAdapter = new MyAdapter(dataList);
47     recyclerView.setAdapter(myAdapter);
48
49     // 可选：设置下拉刷新动画颜色（循环显示）
50     swipeRefresh.setColorSchemeResources(
51         android.R.color.holo_blue_light,
52         android.R.color.holo_red_light,
53         android.R.color.holo_green_light
54     );
55 }
56
57 // 初始化第一页数据
58 private void initData() {
59     loadData(page);
60 }
61
62 // 下拉刷新监听
63 private void setRefreshListener() {
64     swipeRefresh.setOnRefreshListener(new
SwipeRefreshLayout.OnRefreshListener() {
65         @Override
66         public void onRefresh() {
67             // 下拉刷新 → 重置为第一页，清空旧数据
68             page = 1;
69             dataList.clear();
70             loadData(page); // 重新加载第一页
71             swipeRefresh.setRefreshing(false); // 加载完成后，关闭刷新动画
72         }
73     });
74 }
75
76 // 上拉加载监听（滑动到列表底部触发）
77 private void setLoadMoreListener() {
78     recyclerView.addOnScrollListener(new RecyclerView.OnScrollListener() {
79         @Override
80         public void onScrolled(RecyclerView recyclerView, int dx, int dy) {
81             super.onScrolled(recyclerView, dx, dy);
```

```
82         LinearLayoutManager layoutManager = (LinearLayoutManager)
            recyclerView.getLayoutManager();
83
84         // 触发上拉加载的条件:
85         // 1. 向下滑动 (dy>0)
```

原生实现总结

- 优点：无第三方依赖、体积小、适配性强、系统原生风格
- 缺点：需手动处理加载状态（如加载中动画、无数据提示）、下拉刷新样式固定（仅转圈）、不支持侧滑删除等复杂功能

二、常用第三方库（简化开发，功能丰富）

原生方案适合简单需求，复杂场景（自定义刷新样式、侧滑、快速适配）推荐用成熟第三方库，以下是 3 个最实用的：

1. SmartRefreshLayout（最主流：下拉+上拉全能）

核心特点

- 支持自定义刷新/加载样式（图片、动画、进度条等）
- 兼容 RecyclerView、ScrollView、WebView 等所有可滚动控件
- 自带加载中、无数据、错误重试等状态

极简用法（Java）

1. 添加依赖（build.gradle）：

Code block

```
1  // 核心库
2  implementation 'com.scwang.smart:refresh-layout-kernel:2.0.5'
3  // 经典刷新头（可选，也可自定义）
4  implementation 'com.scwang.smart:refresh-header-classics:2.0.5'
5  // 经典加载尾（可选）
6  implementation 'com.scwang.smart:refresh-footer-classics:2.0.5'
```

2. XML 布局（替换原生 SwipeRefreshLayout）：

Code block

```
1  <com.scwang.smart.refresh.layout.SmartRefreshLayout
2      android:id="@+id/smart_refresh"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent">
```

```

5
6         <androidx.recyclerview.widget.RecyclerView
7             android:id="@+id/recycler_view"
8             android:layout_width="match_parent"
9             android:layout_height="match_parent"/>
10
11     </com.scwang.smart.refresh.layout.SmartRefreshLayout>

```

3. Java 逻辑（监听刷新/加载）：

Code block

```

1  SmartRefreshLayout smartRefresh = findViewById(R.id.smart_refresh);
2
3  // 下拉刷新
4  smartRefresh.setOnRefreshListener(refreshLayout -> {
5      page = 1;
6      dataList.clear();
7      loadData(page);
8      refreshLayout.finishRefresh(); // 结束刷新（必须调用）
9  });
10
11 // 上拉加载
12 smartRefresh.setOnLoadMoreListener(refreshLayout -> {
13     page++;
14     loadData(page);
15     refreshLayout.finishLoadMore(); // 结束加载（必须调用）
16 });

```

2. BRVAH（万能适配器：简化 RecyclerView 代码）

核心特点

- 无需写 ViewHolder、onBind 等重复代码，一行绑定数据
- 自带上拉加载、侧滑删除、多类型布局、空数据提示
- 可与 SmartRefreshLayout 搭配使用

极简用法（Java）

1. 添加依赖（build.gradle）：

Code block

```

1  implementation 'com.github.CymChad:BaseRecyclerViewAdapterHelper:3.0.4'

```

2. 适配器（无需写 ViewHolder）：

Code block

```
1  import com.chad.library.adapter.base.BaseQuickAdapter;
2  import com.chad.library.adapter.base.viewholder.BaseViewHolder;
3
4  public class MyBRVAHAdapter extends BaseQuickAdapter<String, BaseViewHolder> {
5
6      // 构造方法：传入布局ID和数据
7      public MyBRVAHAdapter() {
8          super(R.layout.item_list);
9      }
10
11     // 绑定数据（直接用ViewHolder设置）
12     @Override
13     protected void convert(BaseViewHolder helper, String item) {
14         helper.setText(R.id.tv_item, item); // 绑定文本
15     }
16 }
```

3. Activity 中使用（自带上拉加载）：

Code block

```
1  MyBRVAHAdapter adapter = new MyBRVAHAdapter();
2  recyclerView.setAdapter(adapter);
3
4  // 上拉加载（自带加载更多模块）
5  adapter.getLoadMoreModule().setOnLoadMoreListener(() -> {
6      page++;
7      loadData(page);
8      adapter.addData(newData); // 添加新数据
9      adapter.getLoadMoreModule().loadMoreComplete(); // 结束加载
10 });
```

3. Glide + RecyclerView（图片列表专用）

核心特点

- 集成 Glide 图片加载框架，优化列表图片加载（缓存、占位图、预加载）
- 适合图片列表场景（如电商商品、朋友圈）
- 可搭配 SmartRefreshLayout 实现下拉/上拉

极简用法

1. 添加依赖 (build.gradle) :

Code block

```
1 // Glide 图片加载核心库
2 implementation 'com.github.bumptech.glide:glide:4.16.0'
3 annotationProcessor 'com.github.bumptech.glide:compiler:4.16.0'
```

2. 条目布局 (添加 ImageView) :

Code block

```
1 <!-- item_image_list.xml -->
2 <LinearLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     android:layout_width="match_parent"
5     android:layout_height="120dp"
6     android:orientation="horizontal"
7     android:gravity="center_vertical"
8     android:padding="8dp">
9
10    <ImageView
11        android:id="@+id/iv_img"
12        android:layout_width="80dp"
13        android:layout_height="80dp"
14        android:scaleType="centerCrop"/>
15
16    <TextView
17        android:id="@+id/tv_text"
18        android:layout_width="wrap_content"
19        android:layout_height="wrap_content"
20        android:layout_marginStart="8dp"
21        android:textSize="16sp"/>
22
23 </LinearLayout>
```

3. 适配器中加载图片:

Code block

```
1 @Override
2 protected void convert(BaseViewHolder helper, String imgUrl) {
3     // 用Glide加载图片 (占位图+错误图)
4     Glide.with(helper.itemView.getContext())
5         .load(imgUrl) // 图片URL
6         .placeholder(R.drawable.ic_placeholder) // 占位图
```

```
7         .error(R.drawable.ic_error) // 加载失败图
8         .into(helper.getView(R.id.iv_img)); // 绑定到ImageView
9
10     helper.setText(R.id.tv_text, "图片条目");
11 }
```

三、选型建议

1. 简单列表（仅展示+下拉上拉）：原生 RecyclerView + SwipeRefreshLayout
2. 复杂列表（自定义刷新样式、侧滑、多类型）： SmartRefreshLayout + BRVAH
3. 图片列表（需优化图片加载）： Glide + SmartRefreshLayout + BRVAH
4. 快速开发（不想写适配器）：直接用 BRVAH（自带分页+状态管理）

第三方库的核心价值是「封装重复逻辑」，减少模板代码，让你聚焦业务数据，而非 UI 细节～

（注：文档部分内容可能由 AI 生成）